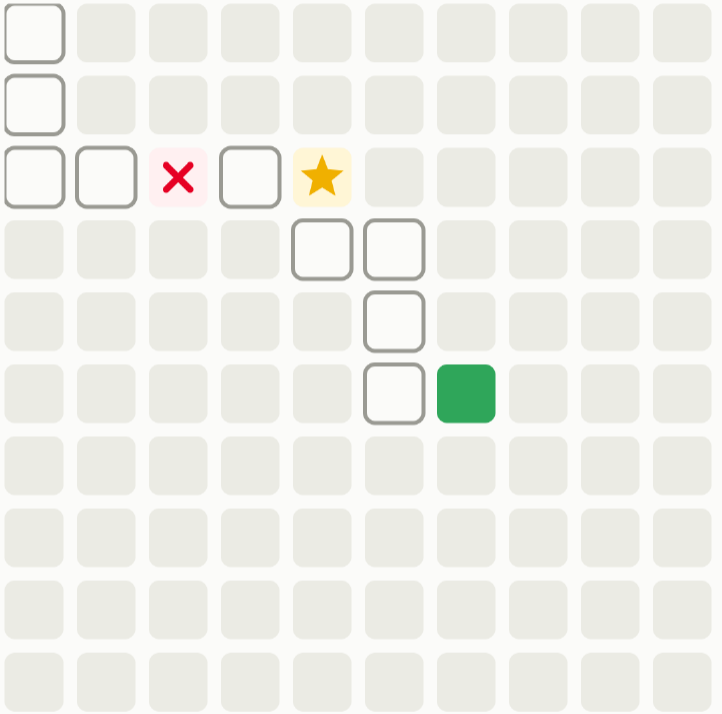
2026. 11. 06 (금)

응용 8-7-1

보물 찾기 게임 Ver2.0

원본 분석과 **30가지** 업그레이드



과목명 **게임프로그래밍**

학과 **소프트웨어학과**

학번 **2023764010**

이름 **김민지**

CONTENTS

목차

01 **원본 코드 설명 및 분석**
원본 게임 개요 · 전체 코드 · 주요 함수

02 **원본 코드의 문제점**
치명적 · 동작상 · 구조 · 이식성 문제 14가지

03 **업그레이드 01~30 (6개 파트)**
원본 vs Ver 2.0 비교 · 파트별 코드 · 총 요약 · 전체 코드

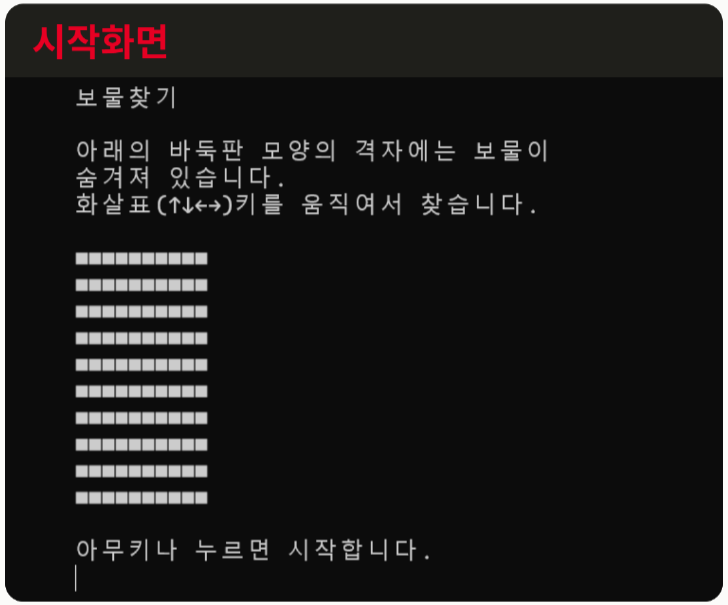
04 **실행화면**
메인 메뉴 · 게임 플레이 · 엔딩 결과

05 **최종 평가**
최종 점수

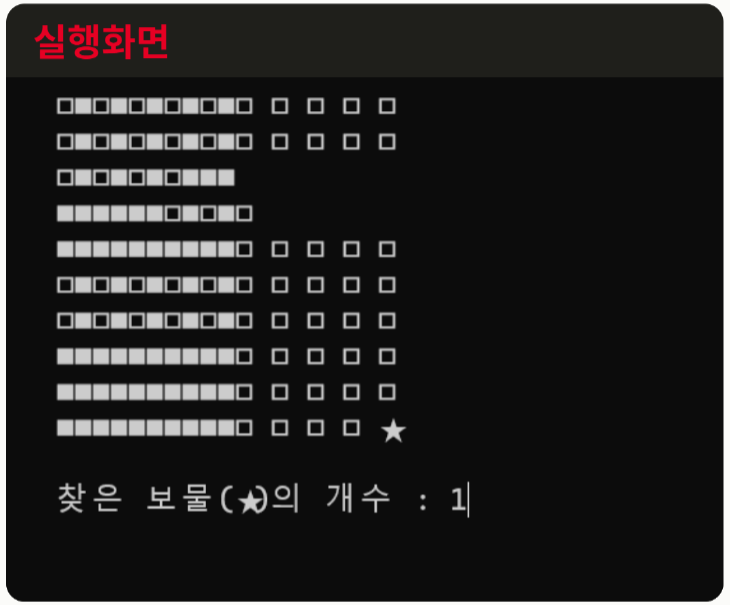
OVERVIEW

원본 게임 개요 및 화면 구성

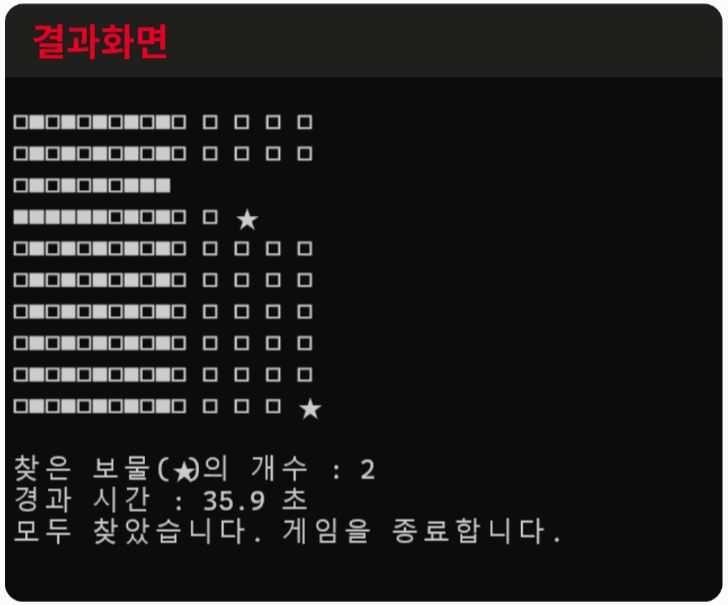
시작화면



실행화면



결과화면



게임 설명

10x10 콘솔 격자 안에 숨은 보물 2개를 방향키로 찾는 게임

종료 조건

보물 2개를 모두 찾으면 종료, 걸린 시간을 초 단위로 출력

진행 흐름

보물 생성 → 규칙 안내 → 게임 루프 → 시간 출력 후 종료

ORIGINAL SOURCE · FULL CODE

원본 전체 소스 코드

응용8_7_1.cpp 157줄

```c
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#include <windows.h>
#include <time.h>

void display_rule(void);
void gotoxy(int x, int y);
void make_treasure(int tx[], int ty[]);
void display_map(int matrix[][10], int tx[], int ty[]);
void basic_map(void);
void move_arrow_key(char key, int *x1, int *y1, int x_b
    , int y_b);
void game_control(int tx[], int ty[]);

int main(void)
{
    int tx[2], ty[2];
    int matrix[10][10]={0};
    clock_t start, end;
    double pst;
    srand(time(NULL));
    make_treasure(tx, ty);
    display_rule();
    start=clock();
    system("cls");
    game_control(tx, ty);
    end=clock();
    pst=(double)(end-start)/CLK_TCK;
    gotoxy(1, 13);
    printf("경과 시간 : %.1f 초" , pst);
    gotoxy(1, 14);
    printf("모두 찾았습니다. 게임을 종료합니다. \n"  );
    getch();
    return 0;
}

void display_rule(void)
{
    gotoxy(1, 1);
    printf("보물찾기" );
    gotoxy(1,3);
    printf("아래의 바둑판 모양의 격자에는 보물이 \n"  );
    gotoxy(1,4);
    printf("숨겨져 있습니다. \n" );
    gotoxy(1,5);
    printf("화살표(↑↓←→)키를 움직여서 찾습니다. \n"
    );
    gotoxy(1,7);
    basic_map();
    printf("\n아무키나 누르면 시작합니다. \n"  );
    getch();
}

void basic_map(void)
{
    int i, j;
    for(i=0;i<10;i++)
    {
        for(j=0;j<10;j++)
            printf("■");
        printf("\n");
    }
}


void make_treasure(int tx[], int ty[])
{
    int i;
    for(i=0;i<2;i++)
    {
        do
        {
            tx[i]=rand()%11; //보물의 x좌표
            ty[i]=rand()%11; //보물의 y좌표
        }while((tx[i]<=1) || (ty[i]<=1));
     }
}



void gotoxy(int x, int y)
{
   COORD Pos = {x - 1, y - 1};
   SetConsoleCursorPosition(GetStdHandle(STD_OUTPUT_HAN
    DLE), Pos);
}

void display_map(int matrix[][11], int tx[], int ty[])
{
    int i, j;
    basic_map();
    for(i=1;i<=10;i++)
        for(j=1;j<=10;j++)
            if (matrix[i][j]==1)
            {
                gotoxy(2*i-1, j);
                printf("□");
            }
            else if (matrix[i][j]==2)
            {
                gotoxy(2*i-1, j);
                printf("★");
            }
}
void move_arrow_key(char key, int *x1, int *y1, int x_b
    , int y_b)
{
    switch(key)
    {
    case 72:  //위쪽(상) 방향의 화살표 키 입력
        *y1=*y1-1;
        if (*y1<1)  *y1=1; //y좌표의 최소값
        break;
    case 75:  //왼쪽(좌) 방향의 화살표 키 입력
        *x1=*x1-2;            //x좌표의 감소값
        if (*x1<1)  *x1=1; //x좌표의 최소값
        break;
    case 77:  //오른쪽(우) 방향의 화살표 키 입력
        *x1=*x1+2;              //x좌표의 증가값
        if (*x1>x_b)  *x1=x_b; //x좌표의 최대값
        break;
    case 80:  //아래쪽(하) 방향의 화살표 키 입력
        *y1=*y1+1;
        if (*y1>y_b)  *y1=y_b; //y좌표의 최대값
        break;
    default:
        return;
    }
}

void game_control(int tx[], int ty[])
{
    char key;
    int i, j, count=0;
    int x=1, y=1;
    int matrix[11][11]={0};
    do
    {
        gotoxy(x, y);
        printf("□");
        matrix[(x+1)/2][y]=1;

        for(i=0;i<2;i++)
            if (((x+1)/2==tx[i]) && (y==ty[i]))
                matrix[(x+1)/2][y]=2;

        gotoxy(1, 1);
        display_map(matrix, tx, ty);

        count=0;
        for(i=1;i<=10;i++)
            for(j=1;j<=10;j++)
                if (matrix[i][j]==2)
                    count=count+1;
        gotoxy(1,12);
        printf("찾은 보물(★)의 개수 : %d" , count);
        key=getch();
        move_arrow_key(key, &x, &y, 19, 10);
    }while(count<2);
}
```

ORIGINAL SOURCE

원본 소스 코드와 main() 실행 흐름

main() 응용8_7_1.cpp 15~32행

```
15  int main(void) {
16    int tx[2], ty[2]; int matrix[10][10]={0};
17    clock_t start, end; double pst;
18    srand(time(NULL));
19    make_treasure(tx, ty);
20    display_rule();
21    start = clock();
22    system("cls");
23    game_control(tx, ty);
24    end = clock();
25    pst = (double)(end-start) / CLK_TCK;
26    gotoxy(1, 13);
27    printf("경과 시간 : %.1f 초", pst);
28    gotoxy(1, 14);
29    printf("모두 찾았습니다. 게임을 종료합니다. \n");
30    getch();
31    return 0;
32  }
```

1. 난수 생성

srand(time(NULL))로 매판 다른 난수를 만들고 make_treasure()로 보물 2개 배치

2. 타임 측정

게임 전후 clock() 값의 차이로 소요 시간을 구해 화면에 출력

3. 게임 진행

game_control()에서 키 입력·이동·보물 확인을 반복하다 2개를 모두 찾으면 종료

FUNCTIONS

원본 소스 코드 주요 함수 분석

`make_treasure()`	rand()%11 난수로 보물 2개의 (X, Y) 좌표 지정
`game_control()`	키 입력 감지 및 보물 수집(count) 체크 do-while 루프
`move_arrow_key()`	방향키 값(72, 75, 77, 80)에 따른 좌표 이동·경계 체크
`display_map()`	(x+1)/2 공식으로 콘솔 2:1 비율 보정 및 맵 갱신
`gotoxy()`	SetConsoleCursorPosition 기반 커서 위치 이동

PROBLEMS

원본 코드의 문제점

치명적 문제

- 좌우 이동 시 2칸씩 이동 → 맵 밖으로 이탈
- 보물 좌표 중복 → 무한 루프
- 좌표 범위 오류 → 1행·1열 제외

동작상 문제

- getch() 1회 → 방향키 오인·이중 루프
- 매 입력 100칸 재출력 → 깜빡임
- CLK_TCK 구식 → CLOCKS_PER_SEC
- 현재 위치 표시 없음

구조·가독성 문제

- 화면 좌표·배열 인덱스 혼재
- 함수 선언·정의 배열 크기 불일치
- 미사용 코드 (matrix, tx/ty, default)
- 보물 개수 100칸 전수 조사

이식성·사용성 문제

- Windows 전용 (conio, windows.h, cls)
- 콘솔 커서 깜빡임
- ESC·재시작·이동 횟수 없음

03 · UPGRADE OVERVIEW

원본 vs Ver 2.0 한눈에 보기

BEFORE 원본 · 응용8_7_1.cpp

코드 길이	157줄
함수 (main 제외)	7개
보물	2개

- 방향키 이동만, 흑백 출력에 커서 노출
- 보물 2개, 좌표 2~10에서만 생성
- 보물이 겹치면 끝나지 않는 무한 루프
- 이동 제한·점수 없이 시간만 측정, 1회 후 종료

VS

+30
UPGRADES

AFTER Ver 2.0 · 응용8_7_1.cpp

코드 길이 약 3.8배	604줄
함수 (main 제외) 기능별 모듈화	13개
보물 + 함정 2 · 힌트 3	3개

- 메인 메뉴 · 게임 설명 · 카운트다운
- 색상 · 효과음 · 상단 HUD · 하단 안내
- 보물 3 + 함정 2 + 힌트 3, 이동 75회 제한
- 점수 · 등급 결과 화면, 일시정지 · 재시작

소스 최적화 & 기본 시스템

01 맵 배열 크기 안정화

선언·정의 크기 통일, 1~10 인덱스 안전 접근

BEFORE

```
matrix[][10] // display_map 선언
matrix[][11] // display_map 정의
```

→

AFTER

```
void display_map(int matrix[][11]);
int matrix[11][11] = { 0 };
```

02 맵 간격 및 레이아웃 개선

2:1 문자 비율 보정, HUD 아래로 맵 배치

BEFORE

```
gotoxy(2*i-1, j);
```

→

AFTER

```
gotoxy(2 * i + 3, j + 5);
```

03 보물 위치 중복 생성 방지

겹침으로 인한 무한 루프 해결, 1~10 전체 사용

BEFORE

```
tx[i]=rand()%11;
} while((tx[i]<=1) || (ty[i]<=1));
```

→

AFTER

```
tx[i] = rand() % 10 + 1;
} while (tx[1] == tx[0] && ty[1] == ty[0]);
```

04 표준 클럭 적용

구형 상수 대신 표준 상수 사용

BEFORE

```
pst=(double)(end-start)/CLK_TCK;
```

→

AFTER

```
pst = (double)(end-start) / CLOCKS_PER_SEC;
```

05 소스코드 최적화

미사용 변수 제거, 기능별 함수 모듈화

BEFORE

```
int matrix[10][10]={0}; // 미사용
```

→

AFTER

```
// 삭제
display_main_menu(); countdown(); ...
```

UPGRADE 06~10 · 2/6

입력 버퍼, UI & HUD

06 확장 방향키 입력 처리

2바이트 방향키를 한 번에 처리해 깜빡임 해결

BEFORE

```
key=getch(); // 224가 먼저 읽힘
```

→

AFTER

```
if (key == -32 || key == 0)
    key = getch();
```

07 메인 메뉴 시스템

시작 · 설명 · 종료를 고르는 메인 화면

BEFORE

```
display_rule(); // 규칙 후 바로 시작
```

→

AFTER

```
int menu_choice = display_main_menu();
```

08 실시간 상단 HUD 구축

좌표 · 이동 횟수 · 힌트를 상단에 고정 표시

BEFORE

원본에 없던 기능

→

AFTER

```
printf(" [위치] X:%2d, Y:%2d | ", ...);
printf("[이동] %2d / %2d 회", ...);
```

09 플레이어 시각화

현재 위치 &(초록), 지나온 칸 □(회색)

BEFORE

```
printf("□"); // 지나온 칸만 표시
```

→

AFTER

```
set_color(10); printf("&");
```

10 콘솔 텍스트 색상 적용

set_color()로 요소마다 다른 색상

BEFORE

원본에 없던 기능

→

AFTER

```
SetConsoleTextAttribute(hOut, (WORD)color);
```

결과, 알고리즘, 사운드 & 점수

11 승리 / 패배 결과 리포트

시간 · 이동 · 보물 · 점수 · 등급 리포트

BEFORE

```
printf("경과 시간 : %.1f 초", pst);
```

→

AFTER

```
display_result(found_count, move_count, pst);
```

12 거리 힌트 시스템

H키로 남은 보물까지 맨해튼 거리 안내

BEFORE

원본에 없던 기능

→

AFTER

```
dist = abs(grid_x - tx[t])
     + abs(grid_y - ty[t]);
```

13 Beep sound 효과음

시작 · 보물 · 함정 · 클리어 · 오버 효과음

BEFORE

원본에 없던 기능

→

AFTER

```
Beep(784, 120); Beep(1047, 200); // 보물
```

14 재시작 및 종료 선택

결과 화면에서 R 재시작 / Q 종료

BEFORE

```
getch(); return 0; // 한 판 후 종료
```

→

AFTER

```
if (select == 'r' || select == 'R') break;
```

15 최종 점수 산출 공식

S+ · S · A · B · F 등급 분기

BEFORE

원본에 없던 기능

→

AFTER

```
score = (remaining_moves * 20)
      - (int)(pst * 2);
```

UPGRADE 16~20 · 4/6

조작, 일시정지, 함정 & 보물확장

16 WASD 다중 조작 지원

방향키와 WASD 대·소문자 모두 인식

BEFORE

```
case 72: // 방향키만
```

→

AFTER

```
case 72: case 'w': case 'W':
```

17 ESC 일시정지 메뉴

1: 계속하기 / 2: 게임 종료

BEFORE

원본에 없던 기능

→

AFTER

```
if (key == 27) { // 일시정지
  if (select == '2') exit(0); }
```

18 최대 이동 제한 설정

75회를 모두 쓰면 GAME OVER

BEFORE

원본에 없던 기능

→

AFTER

```
int max_moves = 75;
if (move_count >= max_moves) break;
```

19 무작위 함정 시스템

함정 X 2개, 밟으면 3초간 이동 불가

BEFORE

원본에 없던 기능

→

AFTER

```
make_traps(hx, hy, tx, ty);
Sleep(3000); // 3초 이동 불가
```

20 보물 수집 확장

보물 2개 → 3개로 난이도 개편

BEFORE

```
int tx[2], ty[2];
```

→

AFTER

```
int tx[3], ty[3];
```

UPGRADE 21~25 · 5/6

비주얼, 가이드 & 예외처리

21 콘솔 커서 숨김 처리

깜빡이는 커서와 잔상 제거

BEFORE

```
원본에 없던 기능
```

→

AFTER

```
CONSOLE_CURSOR_INFO cursorInfo
                    = { 1, FALSE };
```

22 기호 및 맵 비주얼 정밀화

★ · X · & 기호마다 다른 색상

BEFORE

```
printf("★"); // 흑백 출력
```

→

AFTER

```
set_color(14); printf("★");
set_color(12); printf("X ");
```

23 조작 가이드 표시

화면 하단 20행에 조작 안내 고정

BEFORE

```
원본에 없던 기능
```

→

AFTER

```
gotoxy(1, 20); printf("[조작] 이동: ...");
```

24 잘못된 키 예외 처리

잘못된 입력 시 경고 메시지 출력

BEFORE

```
default: return; // 그냥 무시
```

→

AFTER

```
default: *valid_move = 0;
printf("[경고] 잘못된 입력입니다!");
```

25 실시간 보물 카운터

상단 HUD에 ★ ☆ 진행도로 표시

BEFORE

```
printf("...의 개수 : %d", count);
```

→

AFTER

```
printf("★ ☆ ☆ "); // 진행도
```

UPGRADE 26~30 · 6/6

시간, 턴보정, 카운트다운 & 힌트제한

	BEFORE	AFTER
26 시간 분/초 단위 환산 소요 시간을 X분 Y초로 표시	`printf("경과 시간 : %.1f 초", pst);`	→ `minutes = (int)pst / 60;` `seconds = (int)pst % 60;`
27 무효 이동 횟수 제외 벽에 막힌 이동은 횟수에서 제외	원본에 없던 기능	→ `if (prev_x == *x1 && prev_y == *y1)` `  *valid_move = 0;`
28 시작 전 카운트다운 연출 3 · 2 · 1 · START! + 효과음	`display_rule(); start=clock();`	→ `countdown(); // 3, 2, 1, START!`
29 플레이어 실시간 좌표 추적 HUD 1행에 현재 격자 좌표 표시	원본에 없던 기능	→ `printf(" [위치] X:%2d, Y:%2d",` `  grid_x, grid_y);`
30 힌트 사용 횟수 제한 판당 힌트 최대 3회	원본에 없던 기능	→ `if (hint_count > 0) { hint_count--; ... }`

SUMMARY

총 업그레이드 요약

UPGRADE 01~05 기본 시스템

01 배열 안정화
02 간격 개선
03 중복 방지
04 표준 클럭
05 코드 최적화

UPGRADE 06~10 입력·UI·HUD

06 확장키 입력
07 메인 메뉴
08 상단 HUD
09 플레이어 표시
10 텍스트 색상

UPGRADE 11~15 결과·점수·사운드

11 결과 리포트
12 거리 힌트
13 Beep 효과음
14 재시작/종료
15 점수 공식

UPGRADE 16~20 조작·함정·확장

16 WASD 조작
17 ESC 일시정지
18 75턴 제한
19 함정 시스템
20 보물 3개

UPGRADE 21~25 비주얼·예외처리

21 커서 숨김
22 기호 정밀화
23 조작 가이드
24 예외 처리
25 보물 카운터

UPGRADE 26~30 시간·힌트제한

26 분/초 환산
27 무효 이동 제외
28 카운트다운
29 좌표 추적
30 힌트 3회

Ver 2.0 전체 소스 코드 ①

응용8_7_1.cpp

1~203행 / 604줄

```c
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#include <windows.h>
#include <time.h>
#include <math.h>

void gotoxy(int x, int y);
void set_color(int color);
void set_window_title(void);

int display_main_menu(void);
void display_rule(void);
void countdown(void);
void display_result(int count, int move_count, double pst);

void basic_map(void);
void display_map(int matrix[][11]);
void make_treasure(int tx[], int ty[]);
void make_traps(int hx[], int hy[], int tx[], int ty[]);
void move_arrow_key(char key, int *x1, int *y1, int x_b, int y_b, int *valid_mo
    ve);
int game_control(int tx[], int ty[], int hx[], int hy[], int *found_count);

int main(void)
{
    set_window_title();

    CONSOLE_CURSOR_INFO cursorInfo = { 1, FALSE };
    SetConsoleCursorInfo(GetStdHandle(STD_OUTPUT_HANDLE), &cursorInfo);

    while (1)
    {
        int menu_choice = display_main_menu();

        if (menu_choice == 3)
        {
            system("cls");
            set_color(15);
            printf("\n게임을 종료합니다!\n" );
            break;
        }

        if (menu_choice == 2)
        {
            display_rule();
        }

        int tx[3], ty[3];
        int hx[2], hy[2];
        clock_t start, end;
        double pst;
        int found_count = 0;

        srand((unsigned int)time(NULL));
        make_treasure(tx, ty);
        make_traps(hx, hy, tx, ty);

        countdown();

        system("cls");
        start = clock();
        int move_count = game_control(tx, ty, hx, hy, &found_count);
        end = clock();

        pst = (double)(end - start) / CLOCKS_PER_SEC;

        display_result(found_count, move_count, pst);

        set_color(13);
        printf("\n\n\n [ R : 다시 시작 (Restart)  |  Q : 게임 종료 (Quit) ]\n\n
    ");

        int loop_break = 0;
        while (1)
        {
            char select = getch();
            if (select == 'r' || select == 'R')
            {
                break;
            }
            else if (select == 'q' || select == 'Q')
            {
                loop_break = 1;
                break;
            }
        }

        if (loop_break)
        {
            system("cls");
            set_color(15);
            printf("\n게임을 플레이해 주셔서 감사합니다!\n"  );
            break;
        }
    }

    return 0;
}

void gotoxy(int x, int y)
{
    COORD Pos = { (SHORT)(x - 1), (SHORT)(y - 1) };
    SetConsoleCursorPosition(GetStdHandle(STD_OUTPUT_HANDLE), Pos);
}

void set_color(int color)
{
    SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), (WORD)color);
}

void set_window_title(void)
{
    SetConsoleTitle("★ 보물찾기 게임: VER 2.0 ★" );
}

int display_main_menu(void)
{
    while (1)
    {
        system("cls");

        gotoxy(2, 1); set_color(8);
        printf("GAME PROGRAMMING  |  DATE: 2026.11.06  |  VERSION: 2.0");

        set_color(14); gotoxy(9, 3);
        printf("  ___  ___  ___    _   ___  _   _  ___  ___    _  _  _  _  __
    ___");
        gotoxy(9, 4); printf("|_ _|| _ \\| __|  /_\\ / __|| | | || _ \\| __| | |
    || | | | | \\| |_  _|");
        set_color(6);
        gotoxy(9, 5); printf(" | | |   /| _|  / _ \\\\__ \\| |_| ||   /| _|  |
    __ | |_| |\\   | | |  ");
        gotoxy(9, 6); printf("|_| |_|_\\|___|/_/ \\_\\___/ \\___/ |_|_\\|___|
    |_||_|\\___/|_|\\_| |_|  ");

        set_color(11); gotoxy(23, 9);
        printf("★ 숨겨진 보물을 찾아라! ★" );

        set_color(13); gotoxy(5, 11);
        printf("┌──────────────────────────────────────────────────────────
    ──────────┐");
        for (int i = 12; i <= 18; i++) {
            gotoxy(5, i);
            printf("|
            |");
        }
        gotoxy(5, 19); printf("└──────────────────────────────────────────────
    ──────────┘");

        set_color(15); gotoxy(18, 13);
        printf(" [ 1 ]  ▶  게 임 시 작  (Start Game)" );

        gotoxy(18, 15);
        printf(" [ 2 ]  ▶  게 임 설 명  (Game Rules)" );

        gotoxy(18, 17); set_color(12);
        printf(" [ 3 ]  ▶  게 임 종 료  (Exit Game)" );

        set_color(10); gotoxy(16, 21);
        printf("▶ 원하는 메뉴의 번호 [ 1 ~ 3 ] 키를 누르세요: "  );

        char ch = getch();
        if (ch == '1') return 1;
        if (ch == '2') return 2;
        if (ch == '3') return 3;
    }
}

void display_rule(void)
{
    system("cls");

    gotoxy(4, 2); set_color(7);
    printf("═══════════════════════════════════════════════════════════════
    ═══════════════════════════════");
    gotoxy(4, 3); set_color(7);
    printf("                          게임 설명 & 조작 방법                  " );
    gotoxy(4, 4); set_color(7);
    printf("═══════════════════════════════════════════════════════════════
    ═══════════════════════════════");

    gotoxy(6, 6); set_color(11); printf("▶  미션 목표" );
    gotoxy(8, 7); set_color(7); printf("- 10x10 격자에 숨겨진 " );
    set_color(14); printf("3개");
    set_color(7); printf("의 보물을 모두 획득하세요." );
    gotoxy(8, 8); set_color(7); printf("- 주어진 ");
    set_color(14); printf("이동 횟수");
    set_color(7); printf(" 이내에 모아야 탐험 성공!" );

    gotoxy(6, 10); set_color(11); printf("▶  특수 요소" );
    gotoxy(8, 11); set_color(12); printf(" 함정 [X]");
    set_color(7); printf(" : 밟으면 3초간 이동 불가" );
    gotoxy(8, 12); set_color(14); printf(" 힌트 [H]");
    set_color(7); printf(" : 남은 보물까지 거리 탐지 (최대 3회)"  );

    gotoxy(6, 14); set_color(11); printf("▶  점수 산출 공식" );
    gotoxy(8, 15); set_color(7); printf(" (남은 이동 횟수 × 20점) - (소요 시간
     × 2점)");
    gotoxy(8, 16); set_color(13); printf(" S+"); set_color(7); printf(" : 1,100
    점 이상   ");
    set_color(14); printf(" S"); set_color(7); printf(" : 850점 이상");
    gotoxy(8, 17); set_color(10); printf(" A "); set_color(7); printf(" :   500
    점 이상   ");
    set_color(10); printf(" B"); set_color(7); printf(" : 500점 미만 (실패 시 F
    )");
    // ------------------------------------

    gotoxy(6, 19); set_color(11); printf("▶  조작 방법" );
    gotoxy(8, 20); set_color(15); printf(" 이동 : "); set_color(7); printf("방
    향키  또는  W, A, S, D" );
    gotoxy(8, 21); set_color(15); printf(" 힌트 : "); set_color(7); printf("H
    키 (보물 거리 힌트 차감)" );
    gotoxy(8, 22); set_color(15); printf(" 정지 : "); set_color(7); printf("ESC
     키");

    gotoxy(4, 24); set_color(8);
    printf("----------------------------------------------------------------
    ");
    gotoxy(14, 26); set_color(11);
    printf("▶ 아무 키나 누르면 카운트다운 후 게임이 시작됩니다 ◀"   );
```

Ver 2.0 전체 소스 코드 ②

응용8_7_1.cpp 204~401행 / 604줄

```
    getch();
}

void countdown(void)
{
    system("cls");
    gotoxy(1, 4);
    set_color(15);
    printf("     게임을 준비하는 중입니다...\n\n" );

    for (int i = 3; i >= 1; i--)
    {
        gotoxy(1, 6);
        set_color(11);
        printf("              %d          ", i);
        Sleep(500);
    }

    gotoxy(1, 6);
    set_color(12);
    printf("            START!            ");
    Beep(1047, 300);
    Sleep(300);
}

void display_result(int count, int move_count, double pst)
{
    system("cls");

    int minutes = (int)pst / 60;
    int seconds = (int)pst % 60;

    int remaining_moves = 75 - move_count;
    if (remaining_moves < 0) remaining_moves = 0;
    int score = (remaining_moves * 20) - (int)(pst * 2);
    if (score < 0) score = 0;

    char rank[5];
    int rank_color = 15;

    if (count < 3) {
        strcpy(rank, "F");
        rank_color = 12;
    } else if (score >= 1100) {
        strcpy(rank, "S+");
        rank_color = 13;
    } else if (score >= 850) {
        strcpy(rank, "S");
        rank_color = 14;
    } else if (score >= 500) {
        strcpy(rank, "A");
        rank_color = 10;
    } else {
        strcpy(rank, "B");
        rank_color = 10;
    }

    gotoxy(1, 2); set_color(8);
    printf("──────────────────────────────────────────────────────────────────────────────");

    gotoxy(1, 3);
    if (count >= 3)
    {
        set_color(14);
        printf("                ★ GAME CLEAR : 보물 찾기 성공 ★
            ");
    }
    else
    {
        set_color(12);
        printf("                ★ GAME OVER : 보물 찾기 실패 ★
            ");
    }

    gotoxy(1, 4); set_color(8);
    printf("──────────────────────────────────────────────────────────────────────────────");

    gotoxy(1, 6); set_color(11);
    printf(" [ 보물찾기 결과 리포트 ]" );

    gotoxy(1, 8); set_color(15); printf("   총 소요 시간  :  " );
    set_color(7); printf("%d분 %d초", minutes, seconds);

    gotoxy(1, 9); set_color(15); printf("   총 이동 횟수  :  " );
    set_color(7); printf("%d / 75 회", move_count);

    gotoxy(1, 10); set_color(15); printf("   획득한 보물   :  " );
    set_color(7); printf("%d / 3 개", count);

    gotoxy(1, 12); set_color(14);
    printf("   최종 산출 점수" );
    set_color(15); printf("  :  %d 점 ", score);

    set_color(rank_color);
    printf("[ %s 등급 ]", rank);
}

void basic_map(void)
{
    int i, j;

    for (i = 0; i < 10; i++)
    {
        gotoxy(5, i + 6);
        set_color(15);
        for (j = 0; j < 10; j++)
        {
            printf("■ ");
        }
    }
}

void display_map(int matrix[][11])
{
    int i, j;
    basic_map();
    for (i = 1; i <= 10; i++)
    {
        for (j = 1; j <= 10; j++)
        {
            if (matrix[i][j] == 1)
            {
                gotoxy(2 * i + 3, j + 5);
                set_color(15);
                printf("□");
            }
            else if (matrix[i][j] == 2)
            {
                gotoxy(2 * i + 3, j + 5);
                set_color(14);
                printf("★");
            }
            else if (matrix[i][j] == 3)
            {
                gotoxy(2 * i + 3, j + 5);
                set_color(12);
                printf("X ");
            }
        }
    }
}

void make_treasure(int tx[], int ty[])
{
    for (int i = 0; i < 3; i++)
    {
        do {
            tx[i] = rand() % 10 + 1;
            ty[i] = rand() % 10 + 1;
        } while ((i == 1 && tx[1] == tx[0] && ty[1] == ty[0]) ||
                 (i == 2 && ((tx[2] == tx[0] && ty[2] == ty[0]) || (tx[2] == tx[1] && ty[2] == ty[1]))));
    }
}

void make_traps(int hx[], int hy[], int tx[], int ty[])
{
    for (int i = 0; i < 2; i++)
    {
        do {
            hx[i] = rand() % 10 + 1;
            hy[i] = rand() % 10 + 1;
        } while ((hx[i] == tx[0] && hy[i] == ty[0]) ||
                 (hx[i] == tx[1] && hy[i] == ty[1]) ||
                 (hx[i] == tx[2] && hy[i] == ty[2]) ||
                 (hx[i] == 1 && hy[i] == 1) ||
                 (i == 1 && hx[1] == hx[0] && hy[1] == hy[0]));
    }
}

void move_arrow_key(char key, int *x1, int *y1, int x_b, int y_b, int *valid_move)
{
    *valid_move = 1;
    int prev_x = *x1;
    int prev_y = *y1;

    switch (key)
    {
    case 72: case 'w': case 'W':
        *y1 = *y1 - 1;
        if (*y1 < 1) *y1 = 1;
        break;
    case 75: case 'a': case 'A':
        *x1 = *x1 - 2;
        if (*x1 < 1) *x1 = 1;
        break;
    case 77: case 'd': case 'D':
        *x1 = *x1 + 2;
        if (*x1 > x_b) *x1 = x_b;
        break;
    case 80: case 's': case 'S':
        *y1 = *y1 + 1;
        if (*y1 > y_b) *y1 = y_b;
        break;
    default:
        *valid_move = 0;
        gotoxy(1, 22); set_color(12);
        printf("[경고] 잘못된 입력입니다! (방향키, WASD, H, ESC만 가능)
            ");
        return;
    }
```

Ver 2.0 전체 소스 코드 ③

응용8_7_1.cpp

402~604행 / 604줄

```
    if (prev_x == *x1 && prev_y == *y1) {
        *valid_move = 0;
    }
    else {
        gotoxy(1, 22);
        printf("
                ");
    }
}

int game_control(int tx[], int ty[], int hx[], int hy[], int *found_count)
{
    char key;
    int i, j, count = 0;
    int move_count = 0;
    int max_moves = 75;
    int hint_count = 3;
    int x = 1, y = 1;
    int matrix[11][11] = { 0 };

    display_map(matrix);

    do
    {
        int grid_x = (x + 1) / 2;
        int grid_y = y;

        if (matrix[grid_x][grid_y] == 0)
        {
            matrix[grid_x][grid_y] = 1;

            for (i = 0; i < 3; i++)
            {
                if (grid_x == tx[i] && grid_y == ty[i])
                {
                    matrix[grid_x][grid_y] = 2;

                    Beep(784, 120);
                    Beep(1047, 200);
                }
            }

            for (i = 0; i < 2; i++)
            {
                if (grid_x == hx[i] && grid_y == hy[i])
                {
                    matrix[grid_x][grid_y] = 3;

                    gotoxy(1, 22);
                    set_color(12);
                    printf("[함정] 발이 묶였습니다! (3초간 이동할 수 없습니다)
                            ");
                    Beep(200, 300);
                    Sleep(3000);
                    gotoxy(1, 22);
                    printf("
                            ");
                }
            }
        }

        gotoxy(2 * grid_x + 3, grid_y + 5);
        set_color(10);
        printf("&");

        count = 0;
        for (i = 1; i <= 10; i++)
            for (j = 1; j <= 10; j++)
                if (matrix[i][j] == 2)
                    count++;

        *found_count = count;

        gotoxy(1, 1); set_color(14);
        printf("──────────────────────── 보물찾기 게임 : VER 2.0 ──────
──────────────────────");

        gotoxy(1, 2); set_color(15);
        printf(" [위치] X:%2d, Y:%2d | ", grid_x, grid_y);

        if (max_moves - move_count <= 10) set_color(12);
        else set_color(15);
        printf("[이동] %2d / %2d 회", move_count, max_moves);

        set_color(15);
        printf(" | [힌트] %d / 3 회", hint_count);

        gotoxy(1, 3); set_color(11);
        printf(" [보물 진행도] ");
        set_color(15);
        if (count == 0) printf("☆ ☆ ☆ ");
        else if (count == 1) printf("★ ☆ ☆ ");
        else if (count == 2) printf("★ ★ ☆ ");
        else printf("★ ★ ★ ");
        printf(" (%d / 3개 발견)                              ", count);

        gotoxy(1, 19); set_color(8);
        printf("----------------------------------------------------------
--------");
        gotoxy(1, 20); set_color(7);
        printf("[조작] 이동: 방향키 / WASD  |  힌트: H  |  일시정지: ESC"  );

        if (count >= 3)
        {
            gotoxy(1, 22);
            set_color(10);
            printf(" GAME CLEAR - 모든 보물을 찾아냈습니다!
                    \n");
            Beep(523, 120);
            Beep(659, 120);
            Beep(784, 120);
            Beep(1047, 350);
            break;
        }

        if (move_count >= max_moves)
        {
            gotoxy(1, 22);
            set_color(12);
            printf("GAME OVER - 이동 횟수를 모두 소진했습니다!
                    \n");
            Beep(300, 250);
            Beep(200, 400);
            break;
        }

        key = getch();

        gotoxy(2 * grid_x + 3, grid_y + 5);
        if (matrix[grid_x][grid_y] == 2) {
            set_color(14);
            printf("★");
        }
        else if (matrix[grid_x][grid_y] == 3) {
            set_color(12);
            printf("X ");
        }
        else {
            set_color(8);
            printf("□");
        }

        if (key == 27)
        {
            gotoxy(1, 22);
            set_color(10);
            printf("[일시정지] 1: 계속하기 / 2: 게임종료
                    ");
            char select = getch();
            if (select == '2')
            {
                exit(0);
            }
            gotoxy(1, 22);
            printf("
                    ");
        }
        else if (key == 'h' || key == 'H')
        {
            if (hint_count > 0)
            {
                hint_count--;

                int target_idx = -1;
                for (i = 0; i < 3; i++)
                {
                    if (matrix[tx[i]][ty[i]] != 2)
                    {
                        target_idx = i;
                        break;
                    }
                }

                gotoxy(1, 22); set_color(11);
                if (target_idx != -1)
                {
                    int dist = abs(grid_x - tx[target_idx]) + abs(grid_y - ty[t
arget_idx]);
                    printf("[힌트] 남은 보물까지 거리 약 %d칸! (남은 힌트: %d /
 3회)        ", dist, hint_count);
                    Beep(800, 150);
                }
            }
            else
            {
                gotoxy(1, 22); set_color(15);
                printf("힌트를 모두 사용하셨습니다! (남은 힌트: 0/3회)
                        ");
                Beep(250, 200);
            }
        }
        else
        {
            int valid_move = 0;
            if (key == -32 || key == 0)
            {
                key = getch();
                move_arrow_key(key, &x, &y, 19, 10, &valid_move);
            }
            else
            {
                move_arrow_key(key, &x, &y, 19, 10, &valid_move);
            }

            if (valid_move)
            {
                move_count++;
            }
        }

    } while (1);

    return move_count;
}
```

최종 실행 화면 ① 메인 메뉴



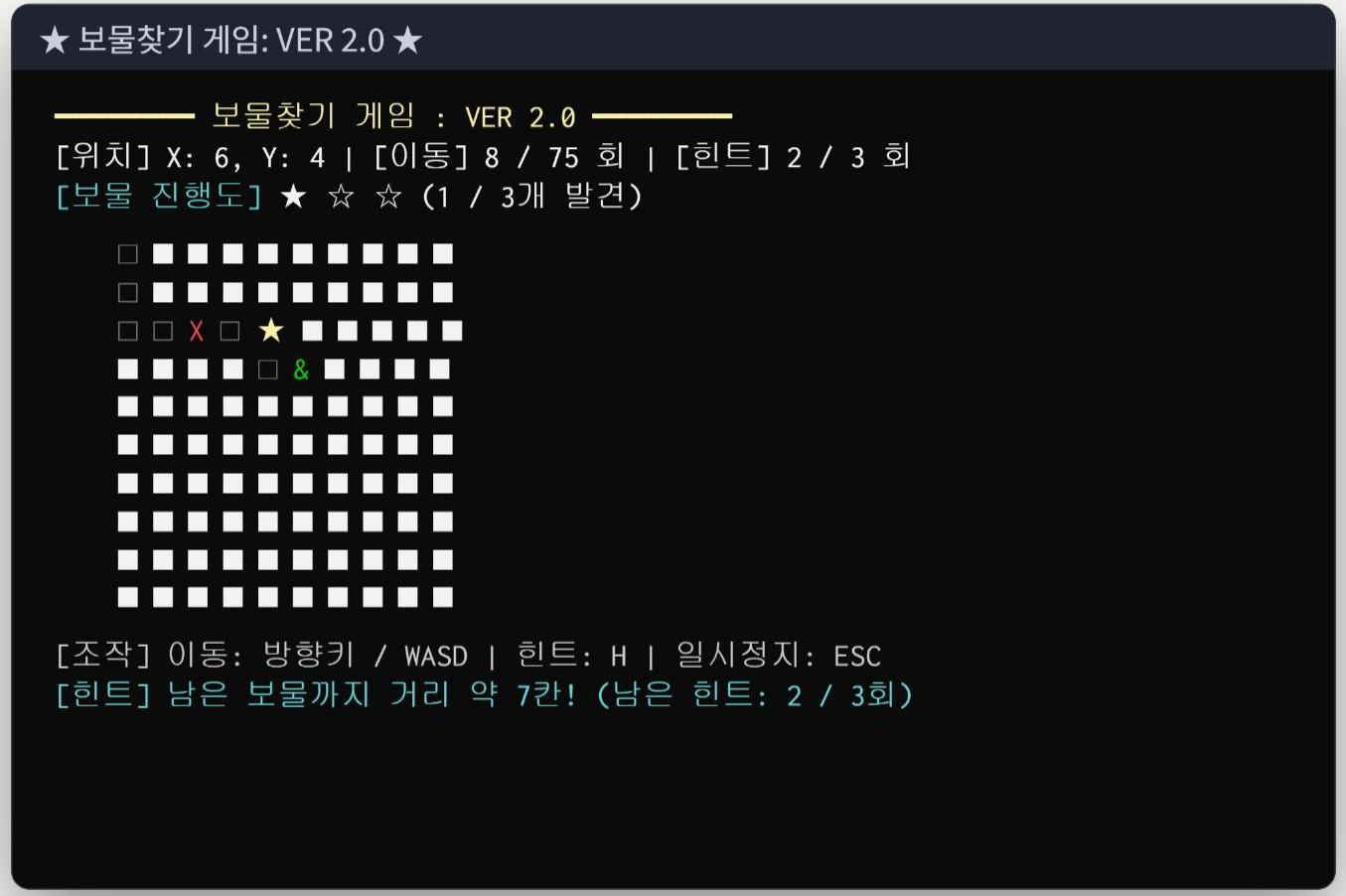
창 제목 + 버전 헤더
SetConsoleTitle로 창 이름 지정, 상단에 날짜·버전 표시

ASCII 아트 로고
노랑 → 진노랑 2단 색상으로 TREASURE HUNT 타이틀

메뉴 박스 1 · 2 · 3
시작 / 설명 / 종료, 숫자키 외 입력은 무시하고 다시 대기

게임 설명 → 카운트다운
2번은 규칙·점수표 화면을 거쳐 3, 2, 1, START!

최종 실행 화면 ② 게임 플레이



상단 HUD

좌표 · 이동 횟수 · 힌트 · 보물 진행도를 매 턴 갱신 (남은 이동 10회 이하면 빨간색)

맵 기호

■ 미탐색 □ 지나온 길 & 플레이어

★ 찾은 보물 X 밟은 함정

하단 안내 · 메시지

조작키 안내를 고정 표시하고, 그 아래 줄에 힌트 · 함정 · 잘못된 입력 메시지 출력

최종 실행 화면 ③ 엔딩 결과

클리어 시 — 보물 3개 모두 발견

```
★ GAME CLEAR : 보물 찾기 성공 ★
[ 보물찾기 결과 리포트 ]
총 소요 시간 : 1분 12초
총 이동 횟수 : 38 / 75 회
획득한 보물 : 3 / 3 개
최종 산출 점수 : 596 점 [ A 등급 ]
[ R : 다시 시작 (Restart) | Q : 게임 종료 (Quit) ]
```

실패 시 — 이동 75회 소진

```
★ GAME OVER : 보물 찾기 실패 ★
[ 보물찾기 결과 리포트 ]
총 소요 시간 : 1분 35초
총 이동 횟수 : 75 / 75 회
획득한 보물 : 2 / 3 개
최종 산출 점수 : 0 점 [ F 등급 ]
[ R : 다시 시작 (Restart) | Q : 게임 종료 (Quit) ]
```

점수 공식 (남은 이동 × 20) − (소요 초 × 2) S+ 1,100↑ · S 850↑ · A 500↑ · B 500 미만 · 보물 3개 미만 F

FINAL SCORE

30 / 30

01–05 06–10 11–15 16–20 21–25 26–30

업그레이드 30개 모두 완료 · 1개당 1점 100%

감사합니다.